

```

1  import java.awt.Cursor;
14
15 /**
16  * View class.
17  *
18  * @author Eric Zhou
19  */
20 @SuppressWarnings("serial")
21 public final class NNCalcView1 extends JFrame implements NNCalcView {
22
23     /**
24      * Controller object registered with this view to observe user-interaction
25      * events.
26      */
27     private NNCalcController controller;
28
29     /**
30      * State of user interaction: last event "seen".
31      */
32     private enum State {
33         /**
34          * Last event was clear, enter, another operator, or digit entry, resp.
35          */
36         SAW_CLEAR, SAW_ENTER_OR_SWAP, SAW_OTHER_OP, SAW_DIGIT
37     }
38
39     /**
40      * State variable to keep track of which event happened last; needed to
41      * prepare for digit to be added to bottom operand.
42      */
43     private State currentState;
44
45     /**
46      * Text areas.
47      */
48     private final JTextArea tTop, tBottom;
49
50     /**
51      * Operator and related buttons.
52      */
53     private final JButton bClear, bSwap, bEnter, bAdd, bSubtract, bMultiply, bDivide,
54         bPower, bRoot;
55
56     /**
57      * Digit entry buttons.
58      */
59     private final JButton[] bDigits;
60
61     /**
62      * Useful constants.
63      */
64     private static final int TEXT_AREA_HEIGHT = 5, TEXT_AREA_WIDTH = 20,
65         DIGIT_BUTTONS = 10, MAIN_BUTTON_PANEL_GRID_ROWS = 4,
66         MAIN_BUTTON_PANEL_GRID_COLUMNS = 4, SIDE_BUTTON_PANEL_GRID_ROWS = 3,
67         SIDE_BUTTON_PANEL_GRID_COLUMNS = 1, CALC_GRID_ROWS = 2, CALC_GRID_COLUMNS = 1;
68
69     /**
70      * index number (avoid magic number).
71      */
72     private static final int INDEX_0 = 0, INDEX_1 = 1, INDEX_2 = 2, INDEX_3 = 3,
73         INDEX_4 = 4, INDEX_5 = 5, INDEX_6 = 6, INDEX_7 = 7, INDEX_8 = 8, INDEX_9 = 9;
74
75     /**
76      * No argument constructor.
77      */

```

```

78 public NNCalcView1() {
79     // Create the JFrame being extended
80
81     /*
82     * Call the JFrame (superclass) constructor with a String parameter to
83     * name the window in its title bar
84     */
85     super("Natural Number Calculator");
86
87     // Set up the GUI widgets -----
88
89     /*
90     * Set up initial state of GUI to behave like last event was "Clear";
91     * currentState is not a GUI widget per se, but is needed to process
92     * digit button events appropriately
93     */
94     this.currentState = State.SAW_CLEAR;
95
96     /*
97     * Create widgets
98     */
99
100    // Set up the GUI widgets -----
101    this.tTop = new JTextArea("", TEXT_AREA_HEIGHT, TEXT_AREA_WIDTH);
102    this.tBottom = new JTextArea("", TEXT_AREA_HEIGHT, TEXT_AREA_WIDTH);
103    this.tTop.setEditable(false);
104    this.tBottom.setEditable(false);
105
106    /*
107    * Create scroll panes for the text areas in case number is long enough
108    * to require scrolling
109    */
110    JScrollPane topScroll = new JScrollPane(this.tTop);
111    JScrollPane bottomScroll = new JScrollPane(this.tBottom);
112
113    /*
114    * Create main button panel
115    */
116    this.bClear = new JButton("Clear");
117    this.bSwap = new JButton("Swap");
118    this.bEnter = new JButton("Enter");
119    this.bAdd = new JButton("+");
120    this.bSubtract = new JButton("-");
121    this.bMultiply = new JButton("*");
122    this.bDivide = new JButton("/");
123    this.bPower = new JButton("Power");
124    this.bRoot = new JButton("Root");
125    this.bDigits = new JButton[DIGIT_BUTTONS];
126    for (int i = 0; i < DIGIT_BUTTONS; i++) { //create the digit button
127        this.bDigits[i] = new JButton(Integer.toString(i));
128    }
129
130    /*
131    * Add the buttons to the main button panel, from left to right and top
132    * to bottom
133    */
134    JPanel mainButtonPanel = new JPanel(new GridLayout(MAIN_BUTTON_PANEL_GRID_ROWS,
135        MAIN_BUTTON_PANEL_GRID_COLUMNS));
136    mainButtonPanel.add(this.bDigits[INDEX_7]); //add the button
137    mainButtonPanel.add(this.bDigits[INDEX_8]);
138    mainButtonPanel.add(this.bDigits[INDEX_9]);
139    mainButtonPanel.add(this.bAdd);
140    mainButtonPanel.add(this.bDigits[INDEX_4]);
141    mainButtonPanel.add(this.bDigits[INDEX_5]);
142    mainButtonPanel.add(this.bDigits[INDEX_6]);

```

```

143 mainButtonPanel.add(this.bSubtract);
144 mainButtonPanel.add(this.bDigits[INDEX_1]);
145 mainButtonPanel.add(this.bDigits[INDEX_2]);
146 mainButtonPanel.add(this.bDigits[INDEX_3]);
147 mainButtonPanel.add(this.bMultiply);
148 mainButtonPanel.add(this.bDigits[INDEX_0]);
149 mainButtonPanel.add(this.bClear);
150 mainButtonPanel.add(this.bEnter);
151 mainButtonPanel.add(this.bDivide);
152
153 /*
154  * Create side button panel
155  */
156 JPanel sideButtonPanel = new JPanel(new GridLayout(SIDE_BUTTON_PANEL_GRID_ROWS,
157     SIDE_BUTTON_PANEL_GRID_COLUMNS));
158 sideButtonPanel.add(this.bSwap);
159 sideButtonPanel.add(this.bPower); //add the side button
160 sideButtonPanel.add(this.bRoot);
161
162 /*
163  * Add the buttons to the side button panel, from left to right and top
164  * to bottom
165  */
166
167 /*
168  * Create combined button panel organized using flow layout, which is
169  * simple and does the right thing: sizes of nested panels are natural,
170  * not necessarily equal as with grid layout
171  */
172 JPanel combinedButtonPanel = new JPanel(new FlowLayout());
173 combinedButtonPanel.add(mainButtonPanel);
174 combinedButtonPanel.add(sideButtonPanel);
175
176 /*
177  * Add the other two button panels to the combined button panel
178  */
179 this.bDivide.setEnabled(false); //initial the disabled button
180 this.bRoot.setEnabled(false);
181 /*
182  * Organize main window
183  */
184 JPanel displayPanel = new JPanel(new GridLayout(2, 1));
185 displayPanel.add(topScroll);
186 displayPanel.add(bottomScroll);
187 this.setLayout(new GridLayout(CALC_GRID_ROWS, CALC_GRID_COLUMNS));
188 this.add(displayPanel);
189 this.add(combinedButtonPanel);
190
191 /*
192  * Add scroll panes and button panel to main window, from left to right
193  * and top to bottom
194  */
195
196 // Set up the observers -----
197 JButton[] allButtons = { this.bClear, this.bSwap, this.bEnter, this.bAdd,
198     this.bSubtract, this.bMultiply, this.bDivide, this.bPower, this.bRoot };
199 for (JButton b : allButtons) {
200     b.addActionListener(this);
201 }
202 for (JButton b : this.bDigits) {
203     b.addActionListener(this);
204 }
205
206 /*
207  * Register this object as the observer for all GUI events

```

```

208    */
209
210    // Set up the main application window -----
211    this.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
212    this.pack();
213    this.setVisible(true);
214
215    /*
216     * Make sure the main window is appropriately sized, exits this program
217     * on close, and becomes visible to the user
218     */
219
220    }
221
222    @Override
223    public void registerObserver(NNCalcController controller) {
224
225        this.controller = controller;
226
227    }
228
229    @Override
230    public void updateTopDisplay(NaturalNumber n) {
231
232        this.tTop.setText(n.toString());
233
234    }
235
236    @Override
237    public void updateBottomDisplay(NaturalNumber n) {
238
239        this.tBottom.setText(n.toString()); //update bottom
240
241    }
242
243    @Override
244    public void updateSubtractAllowed(boolean allowed) {
245
246        this.bSubtract.setEnabled(allowed); //set for subtract
247    }
248
249    @Override
250    public void updateDivideAllowed(boolean allowed) {
251
252        this.bDivide.setEnabled(allowed); //set for divide
253    }
254
255    @Override
256    public void updatePowerAllowed(boolean allowed) {
257
258        this.bPower.setEnabled(allowed); //set for power
259
260    }
261
262    @Override
263    public void updateRootAllowed(boolean allowed) {
264
265        this.bRoot.setEnabled(allowed); //set for root
266
267    }
268
269    @Override
270    public void actionPerformed(ActionEvent event) {
271        /*
272         * Set cursor to indicate computation on-going; this matters only if

```

```

273     * processing the event might take a noticeable amount of time as seen
274     * by the user
275     */
276     this.setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));
277     /*
278     * Determine which event has occurred that we are being notified of by
279     * this callback; in this case, the source of the event (i.e, the widget
280     * calling actionPerformed) is all we need because only buttons are
281     * involved here, so the event must be a button press; in each case,
282     * tell the controller to do whatever is needed to update the model and
283     * to refresh the view
284     */
285     Object source = event.getSource();
286     if (source == this.bClear) {
287         this.controller.processClearEvent();
288         this.currentState = State.SAW_CLEAR;
289     } else if (source == this.bSwap) {
290         this.controller.processSwapEvent();
291         this.currentState = State.SAW_ENTER_OR_SWAP;
292     } else if (source == this.bEnter) {
293         this.controller.processEnterEvent();
294         this.currentState = State.SAW_ENTER_OR_SWAP;
295     } else if (source == this.bAdd) {
296         this.controller.processAddEvent();
297         this.currentState = State.SAW_OTHER_OP;
298     } else if (source == this.bSubtract) {
299         this.controller.processSubtractEvent();
300         this.currentState = State.SAW_OTHER_OP;
301     } else if (source == this.bMultiply) {
302         this.controller.processMultiplyEvent();
303         this.currentState = State.SAW_OTHER_OP;
304     } else if (source == this.bDivide) {
305         this.controller.processDivideEvent();
306         this.currentState = State.SAW_OTHER_OP;
307     } else if (source == this.bPower) {
308         this.controller.processPowerEvent();
309         this.currentState = State.SAW_OTHER_OP;
310     } else if (source == this.bRoot) {
311         this.controller.processRootEvent();
312         this.currentState = State.SAW_OTHER_OP;
313     } else {
314         for (int i = 0; i < DIGIT_BUTTONS; i++) {
315             if (source == this.bDigits[i]) {
316                 switch (this.currentState) {
317                     case SAW_ENTER_OR_SWAP:
318                         this.controller.processClearEvent();
319                         break;
320                     case SAW_OTHER_OP:
321                         this.controller.processEnterEvent();
322                         this.controller.processClearEvent();
323                         break;
324                     default:
325                         break;
326                 }
327                 this.controller.processAddNewDigitEvent(i);
328                 this.currentState = State.SAW_DIGIT;
329                 break;
330             }
331         }
332     }
333     /*
334     * Set the cursor back to normal (because we changed it at the beginning
335     * of the method body)
336     */
337     this.setCursor(Cursor.getDefaultCursor());

```

338 }  
339  
340 }  
341

2000年1月16日星期一 11:11